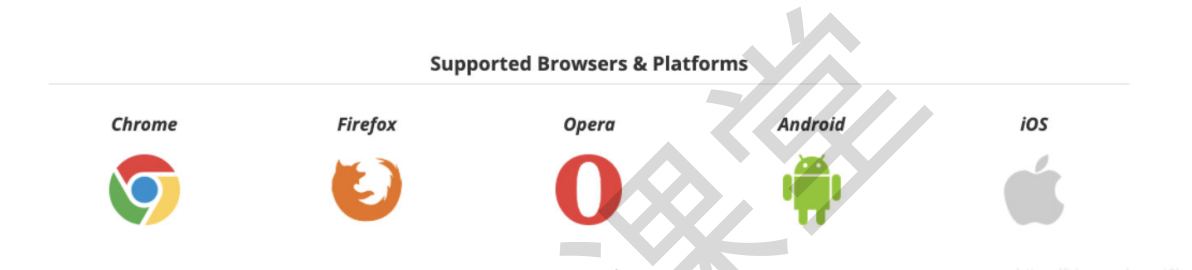


WebRTC是什么

WebRTC是由Google主导的，由一组标准、协议和JavaScript API组成，用于实现浏览器之间（端到端之间）的音频、视频及数据共享。WebRTC不需要安装任何插件，通过简单的JavaScript API就可以使得实时通信变成一种标准功能。现在各大浏览器以及终端已经逐渐加大对WebRTC技术的支持。下图是webrtc官网给出的现在已经提供支持了的浏览器和平台。

WebRTC，名称源自网页即时通信（英语：Web Real-Time Communication）的缩写，是一个支持网页浏览器进行实时语音对话或视频对话的API。它于2011年6月1日开源并在Google、Mozilla、Opera支持下被纳入万维网联盟的W3C推荐标准。谷歌2011年6月3日宣布向开发人员开放WebRTC架构的源代码。这个源代码将根据没有专利费的BSD（伯克利软件发布）式的许可证向用户提供。开发人员可访问并获取WebRTC的源代码、规格说明和工具等。

WebRTC使用安全实时传输协议（Secure Real-time Transport Protocol, SRTP）对RTP数据进行加密，消息认证和完整性以及重播攻击保护。它是一个安全框架，通过加密RTP负载和支持原始认证来提供机密性。WebRTC的安全特性是其可靠性的重要组成部分，其基础全部围绕实时传输协议（Real-time Transport Protocol）进行。



官方资料：

<https://www.w3.org/TR/webrtc/>

相关术语解释

RTP协议(RFC3550)

实时传输协议（RTP）是专为多媒体电话（VoIP，视频会议，远程呈现系统），多媒体流（视频点播，直播）和多媒体广播而设计的网络协议。比如大家想一下：简单的多播音频会议。语音通信通过一个多播地址和一对端口来实现。一个用于音频数据（RTP），另一个用于控制包（RTCP）。

RTP被划分在传输层，它建立在UDP上。同UDP协议一样，为了实现其实时传输功能，RTP也有固定的封装形式。RTP用来为端到端的实时传输提供时间信息和流同步，但并不保证服务质量。服务质量由RTCP来提供。

RTP包的封装格式如下图：

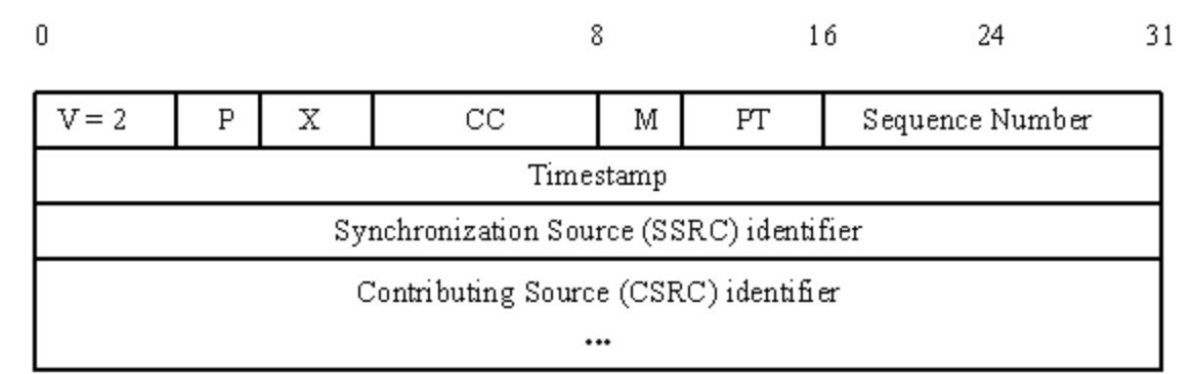


图 2 RTP的头部格式

版本号 (V) : 2比特, 用来标志使用的RTP版本。

填充位 (P) : 1比特, 如果该位置位, 则该RTP包的尾部就包含附加的填充字节。

扩展位 (X) : 1比特, 如果该位置位的话, RTP固定头部后面就跟有一个扩展头部。

CSRC计数器 (CC) : 4比特, 含有固定头部后面跟着的CSRC的数目。

标记位 (M) : 1比特, 该位的解释由配置文档 (Profile) 来承担。

载荷类型 (PT) : 7比特, 标识了RTP载荷的类型。

序列号 (SN) : 16比特, 发送方在每发送完一个RTP包后就将该域的值增加1, 接收方可以由该域检测包的丢失及恢复包序列。序列号的初始值是随机的。

时间戳: 32比特, 记录了该包中数据的第一个字节的采样时刻。在一次会话开始时, 时间戳初始化成一个初始值。即使在没有信号发送时, 时间戳的数值也要随时间而不断地增加 (时间在流逝嘛)。时间戳是去除抖动和实现同步不可缺少的。

同步源标识符(SSRC): 32比特, 同步源就是指RTP包流的来源。在同一个RTP会话中不能有两个相同的SSRC值。该标识符是随机选取的 RFC1889推荐了MD5随机算法。

贡献源列表 (CSRC List) : 0~15项, 每项32比特, 用来标志对一个RTP混合器产生的新包有贡献的所有RTP包的源。由混合器将这些有贡献的SSRC标识符插入表中。SSRC标识符都被列出来, 以便接收端能正确指出交谈双方的身份。

SDP(RFC4566)

SDP全称是Session Description Protocol, 主要用于两个会话实体之间的媒体协商。SDP的主要作用是了解决参与会话的各成员之间能力不对等的问题, 如果参加本次通话的成员都支持高质量的通话, 但是我们没有去进行协议, 为了兼容性, 使用的都是普通质量的通话格式, 这样就很浪费资源了。

SDP定义了会话描述的统一格式, 但并不定义多播地址的分配和SDP消息的传输, 也不支持媒体编码方案的协商, 这些功能均由下层传送协议完成。

SDP协议主要包含:

SDP包括以下一些方面:

- (1) 会话的名称和目的
- (2) 会话存活时间
- (3) 包含在会话中的媒体信息, 包括:
 - 媒体类型(video, audio, etc)
 - 传输协议(RTP/UDP/IP, H.320, etc)
 - 媒体格式(H.261 video, MPEG video, etc)
 - 多播或远端(单播)地址和端口
- (4) 为接收媒体而需的信息(addresses, ports, formats and so on)
- (5) 使用的带宽信息
- (6) 可信赖的接洽信息(Contact information)

//直播编码器生成的sdp文件

```
v=0
o=- 2545495921 1885424500 IN IP4 192.168.225.158
s=111
c=IN IP4 192.168.225.153
b=RR:0
t=0 0
m=video 5088RTP/AVP 96
b=AS:949
a=rtpmap:96 H264/90000
```

```

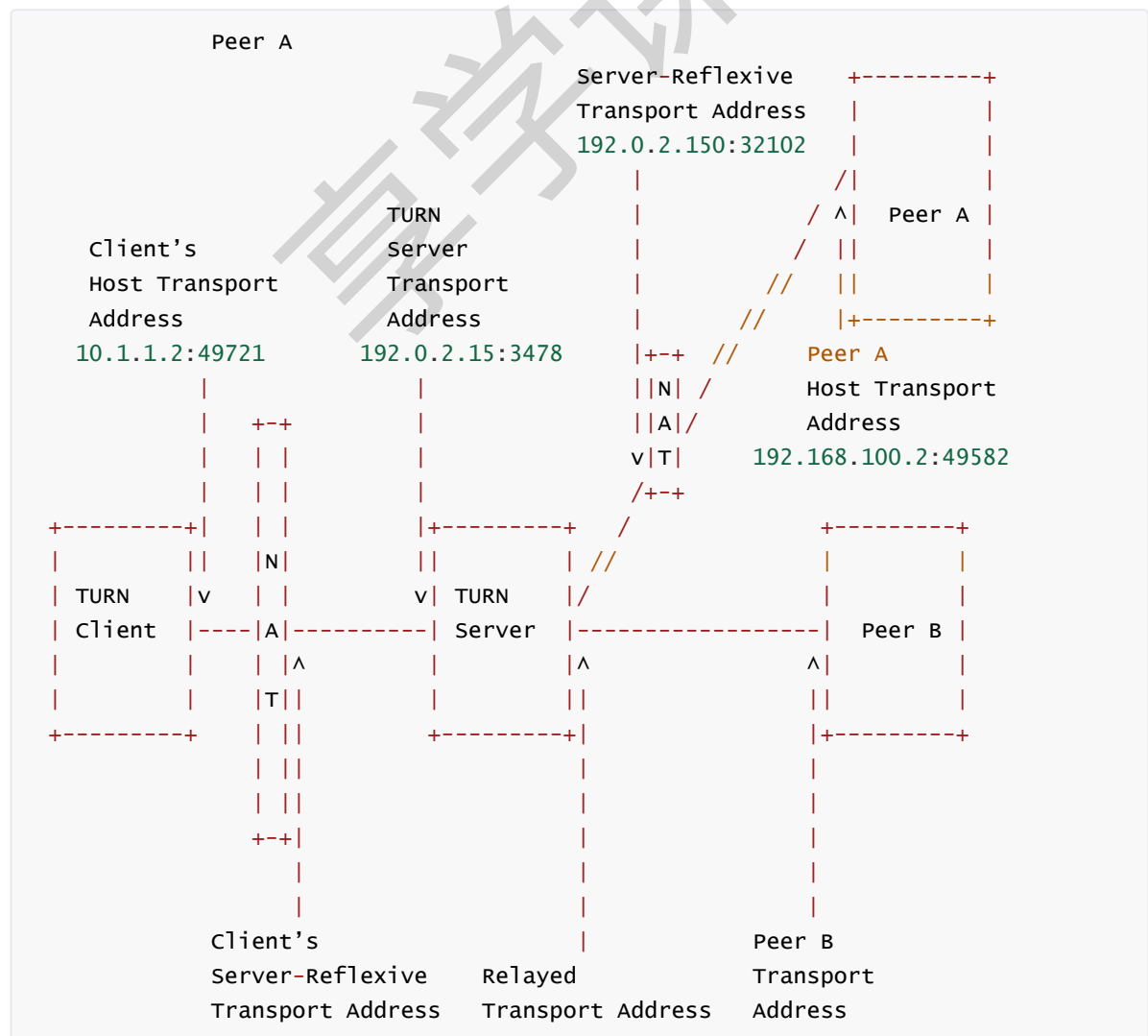
a=fmtp:96 profile-level-id=4D4015;sprop-parameter-
sets=Z01AFZZWCwSbCEiAAAH0AAAw1DBgAHP2A0g1CABQ,a088gA==;packetization-mode=1
a=cliprect:0,0,576,352
a=framerate:25.
a=mpeg4-esid:201
a=x-envivio-verid:0002229D
m=audio 5090 RTP/AVP 97
b=AS:50
a=rtpmap:97 mpeg4-generic/24000/2
a=fmtp:97 profile-level-id=15; config=1310;streamtype=5; ObjectType=64;
mode=AAC-hbr; SizeLength=13; IndexLength=3; IndexDeltaLength=3
a=mpeg4-esid:101
a=lang:eng
a=x-envivio-verid:0002229D

```

TURN(RFC5766)

TURN 协议允许 NAT 或者防火墙后面的对象可以通过 TCP 或者 UDP 接收到数据。TURN 方式解决 NAT 问题的思路与 STUN 相似，是基于私网接入用户通过某种机制预先得到其私有地址对应公网的地址（STUN 方式得到的地址为出口 NAT 上的地址，TURN 方式得到地址为 TURN Server 上的地址），然后在报文负载中所描述的地址信息直接填写该公网地址的方式，实际应用原理也是一样的。

TURN 的全称为 Traversal Using Relay NAT，即通过 Relay 方式穿越 NAT，TURN 应用模型通过分配 TURN Server 的地址和端口作为客户端对外的接受地址和端口，即私网用户发出的报文都要经过 TURN Server 进行 Relay 转发，这种方式应用模型除了具有 STUN 方式的优点外，还解决了 STUN 应用无法穿透对称 NAT（Symmetric NAT）以及类似的 Firewall 设备的缺陷，即无论企业网驻地网出口为哪种类型的 NAT/FW，都可以实现 NAT 的穿透，同时 TURN 支持基于 TCP 的应用，如 H323 协议。

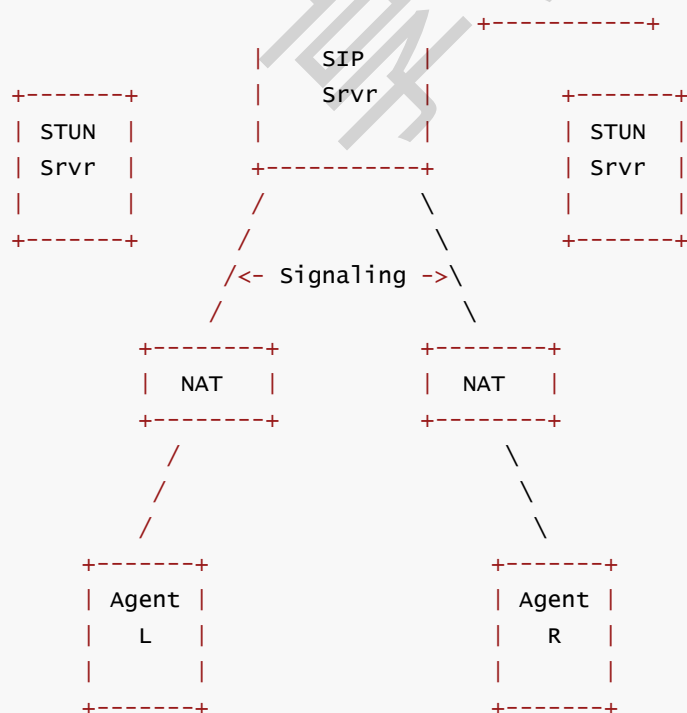


ICE(RFC5245)

ICE全称Interactive Connectivity Establishment：交互式连通建立方式。ICE参照RFC5245建议实现，是一组基于offer/answer模式解决NAT穿越的协议集合。它综合利用现有的STUN，TURN等协议，以更有效的方式来建立会话。客户端侧无需关心所处网络的位置以及NAT类型，并且能够动态的发现最优的传输路径。



//ICE工作环境



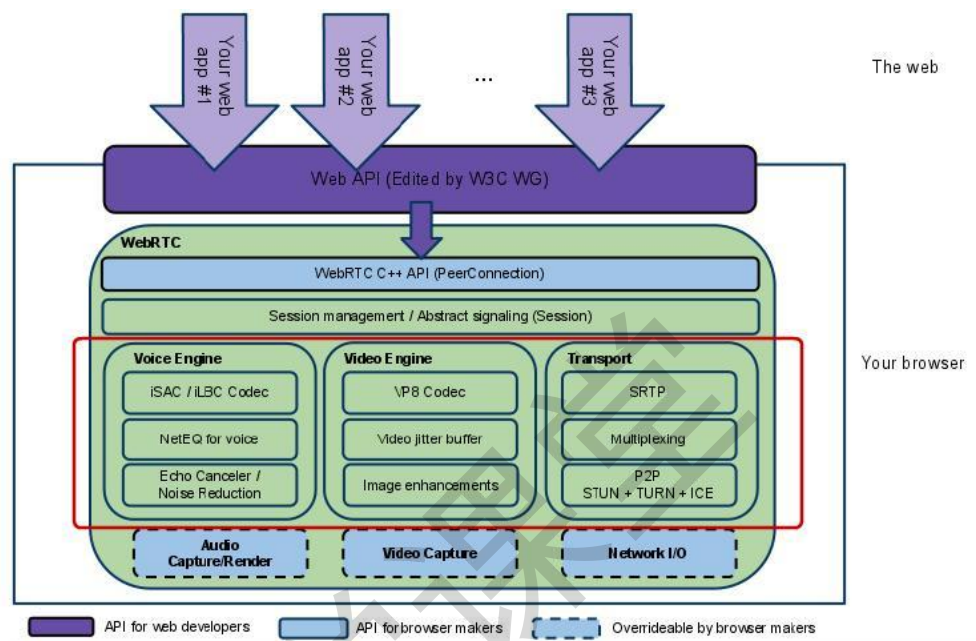
ICE的基本思路是,每个终端都有一系列 传输地址 (包括传输协议,IP地址和端口)的候选,可以用来和其他端点进行通信。

其中可能包括:

- 直接和网络接口联系的传输地址(host address)
- 经过NAT转换的传输地址,即反射地址(server reflective address)
- TURN服务器分配的中继地址(relay address)

可以说,ICE的本质就是一种规范,用于NAT环境之下的多终端通信链路的查找。

WebRTC架构



相关组件介绍:

(1) Your Web App

Web开发者开发的程序, Web开发者可以基于集成WebRTC的浏览器提供的web API开发基于视频、音频的实时通信应用。

(2) Web API

面向第三方开发者的WebRTC标准API (Javascript) , 使开发者能够容易地开发出类似于网络视频聊天的web应用。

(3) WebRTC Native C++ API

本地C++ API层, 使浏览器厂商容易实现WebRTC标准的Web API, 抽象地对数字信号过程进行处理。

(4) Transport / Session

传输/会话层: 会话层组件采用了libjingle库的部分组件实现, 无须使用xmpp/jingle协议。

Muti-Platform API: 这一层 API 是提供给不同平台应用软件开发人员使用的 API, 这些API 都会提供三个功能接口, 分别是MediaStream、RTCPeerConnection和RTCDDataChannel。MediaStream接口用于捕获和存储客户端的实时音视频流, 便于客户端的进行音视频采集和渲染。RTCPeerConnection接口是WebRTC 的核心接口, 它封装了WebRTC连接的管理, 是承载着 WebRTC 连接机制的接口。RTCDDataChannel接口是进行WebRTC 连接数据传输的数据通道接口。多平台API有很多, 上图的Web API和 iOS API 分别是网页和移动端平台的一个代表, 分别是用 JavaScript 和 Objective-C语言进行编写, 让前端开发者和iOS原生应用开发人员可以便捷开发出基于 WebRTC 技术的实时音视频应用程序。

WebRTC C++ API: 这一层的API是提供给浏览器厂商、平台SDK开发者使用的 C++ API, 不同的平台可以通过各自的C++接口调用能力, 对其进行上层封装, 满足跨平台的需求。

Session management/Abstract signaling: 该层是WebRTC的会话层，主要用于进行信令交互和管理 RTCPeerConnection的连接状态。

Voice Engine: 音频引擎模块是一系列音频多媒体处理框架，包括Audio Codecs、NetEQ for voice、Acoustic Echo Canceller (AEC) 和Noise Reduction (NR)。其中，Audio Codecs是音频编解码器，当前 WebRTC 支持ilbc、isac、G711、G722和opus等等；NetEQ for voice 是自适应抖动控制算法以及语音包丢失隐藏算法，用于适应不断变化的网络环境；AEC 是回声消除器，用于实时消除麦克风采集到的回声；NR是噪声抑制器，用于消除与相关 VoIP的某些类型的背景噪音（嘶嘶声、风扇噪声等等）。以上多项音频处理技术集成在一起，使得WebRTC在确保音质优美的同时还减小了缓冲延迟。

Video Engine: 视频引擎模块是一系列视频多媒体处理框架，包括Video Codec、Video Jitter Buffer和Image Enhancement。其中，Video Codec是视频编解码器，当前WebRTC 支持VP8、VP9和H.264编解码；Video Jitter Buffer是视频抖动缓冲器，用于降低由于视频抖动和视频信息包丢失带来的不良影响；Image Enhancement 是图像质量增强模块，用于对摄像头采集回来的图像进行处理，包括明暗度检测、颜色增强、降噪处理等。以上多项视频处理技术集成在一起，使得WebRTC在确保画面优美的同时还提高了流畅度。

Transport: 数据传输模块是WebRTC对音视频进行P2P传输的核心模块，包括SRTP、Multiplexing和P2P。其中，SRTP是基于UDP的安全实时传输协议，为WebRTC中音视频数据提供安全单播和多播功能；Multiplexing 是多路复用技术，采用多路复用技术能把多个信号组合在一条物理信道上进行传输，减少对传输线路的数量消耗；P2P是端对端传输技术，WebRTC 的P2P技术集成了STUN、TURN和ICE，这些都是针对 UDP 的NAT的防火墙穿越方法，是连接有效性的保障。

WebRTC核心通信原理

媒体协商

借助SDP协议完成多媒体信息的交换。

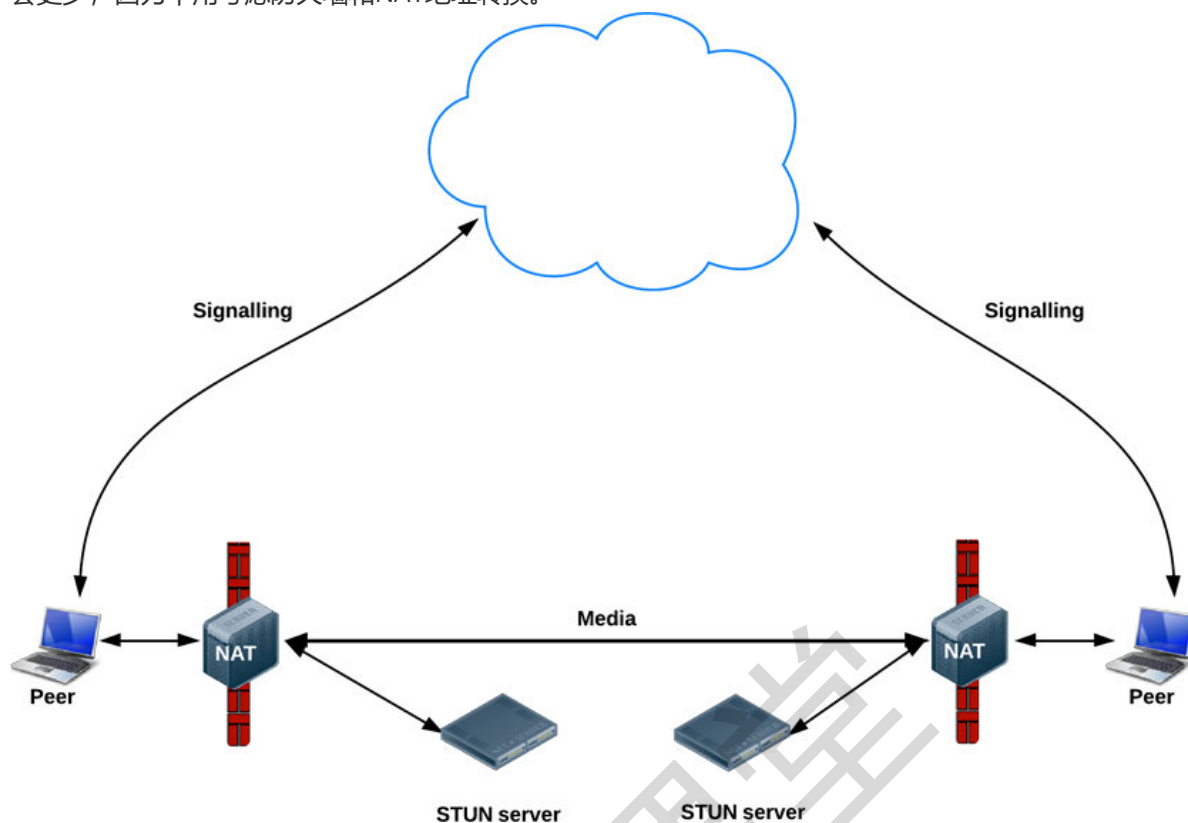
网络交换

思考一个问题：WebRTC可以实现P2P的数据传输，那还需要网络服务器吗？网络服务器主要的作用是什么？

STUN

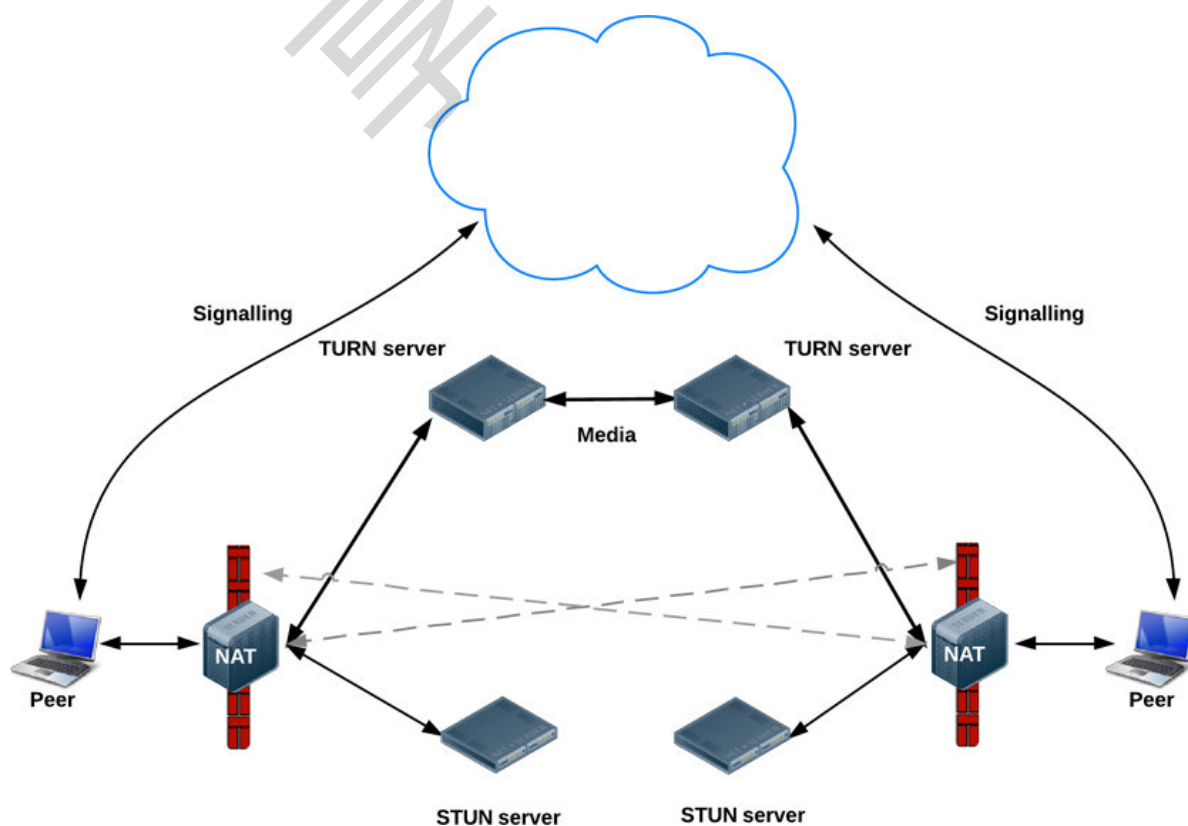
NAT为设备提供内网IP地址，以便在专用本地网络中使用，但是这个地址不能在外网使用。对于WebRTC而言，没有公共地址，点与点之间就无法直接进行通信。为了解决这个问题，WebRTC采用STUN技术。很多内网的设备可以给公网地址发送数据，并不知道公网是用什么的地址来识别自己的，STUN服务器在收到查询请求的时候会告诉这个设备它的公网地址是啥样子的。设备拿到这个地址把这个地址发送给需要建立直接联系的其他设备。

STUN服务器对计算性能和存储要求都不太高，因此相对低规格的STUN服务器可以处理大量请求。根据webrtcstats.com的统计，有86%的WebRTC应用使用STUN成功建立连接，在内网端点之间的呼叫可能会更少，因为不用考虑防火墙和NAT地址转换。



TURN

TURN服务器具有公共地址，因此即使端点位于防火墙或代理之后，也可以与其他端点进行通信。TURN服务器虽然只有这么一个简单的任务——中继流，但与STUN服务器不同，它们本身就消耗了大量带宽。换句话说，TURN服务器需要更强大。RTCPeerConnection尝试通过UDP建立点与点之间的直接通信。如果失败，RTCPeerConnection将转向TCP。如果TCP连接失败，可以将TURN服务器用作回退，在端点之间中继数据。



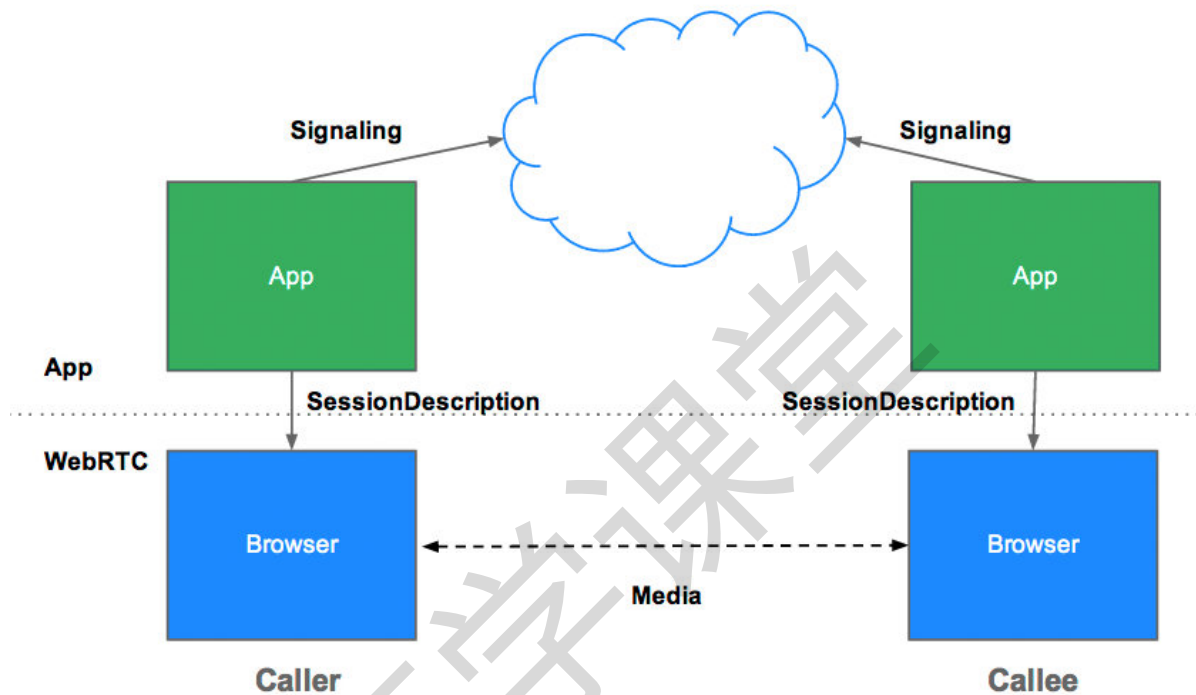
####信令服务器

信令

信令用于协调通信，WebRTC应用开始通话之前，客户端需要交换一些信息（信令）：

- 用于打开或关闭通信的会话控制消息。
- 错误信息。
- 媒体元数据，例如编解码器和编解码器设置，带宽和媒体类型。
- 用于建立安全连接的的密钥信息。
- 主机的IP和端口等网络信息。

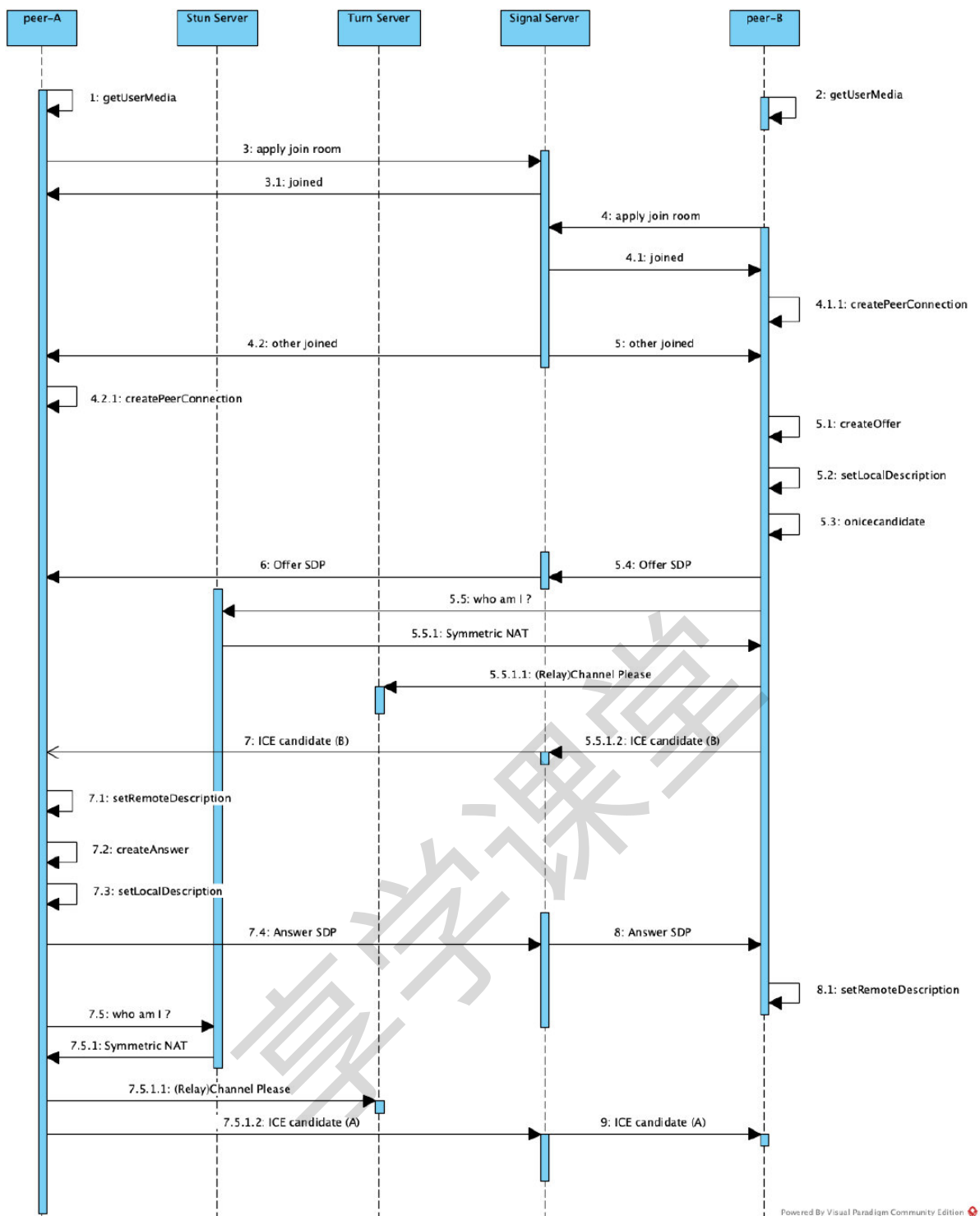
为了避免冗余并提高与已有技术的兼容性，WebRTC标准未规定信令方法和协议。



在WebRTC中需要理解信令服务器的主要作用：

- 业务上的的隔离，比如对于房间的管理，会有类似群的逻辑概念。
- SDP的交换

WebRTC 1V1通话原理



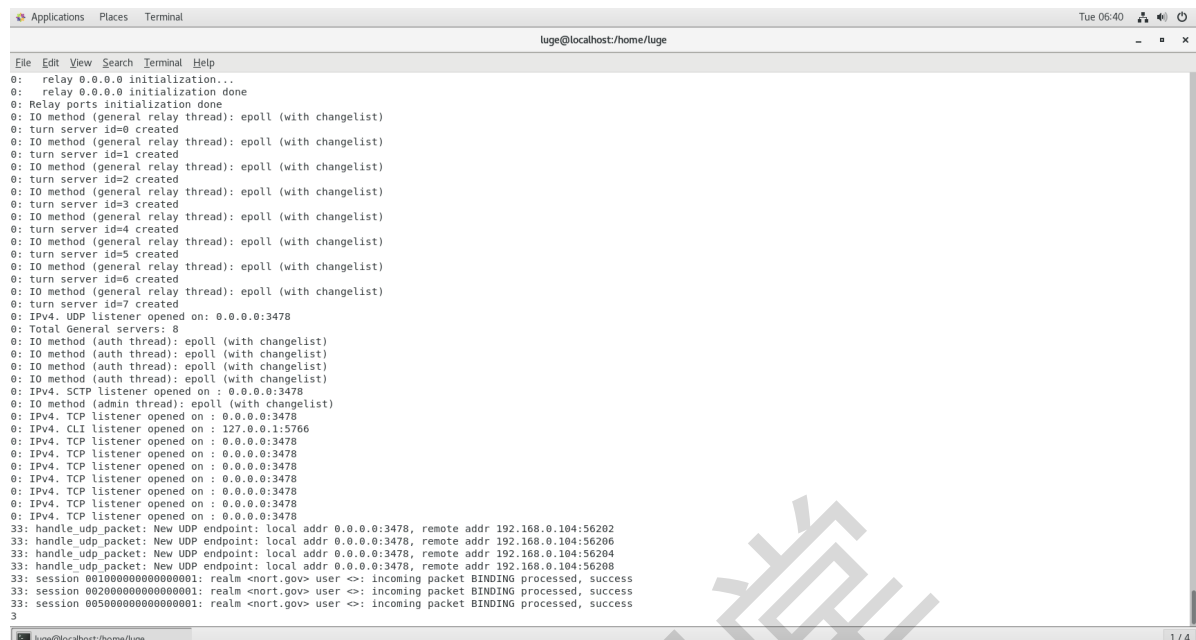
主要过程如下：

- 1、双方先调用getUserMedia打开本机摄像头
- 2、向信令服务器发送加入房间apply_join请求
- 3、信令服务器通知本人加入成功(joined)，同时向其它人广播加入消息(other_joined)
- 4、二端开始创建peerConnection连接
- 5、peerB端创建offer，同时将SDP保存到本机(setLocalDescription)，并通过信令服务器传递到peerA
- 6、peerB在setLocalDescription后，会异步触发“候选网络链路”收集，大致是通过Stun确定自己所有的NAT映射出口，如果Stun返回NAT是“对称型”的，基本上就没法穿透了，会再通过Turn拿到中继reply地址，并通过信令服务器，将网络候选链路信息发到peerA(即：开始网络协商)
- 7、peerA收到的peerB的SDP后，开始回应(createAnswer)，仍然通过信令服务器，将SDP发送到peerB

8、同时peerA也会开始网络候选链路的收集，并将自己的网络信息，通过信令服务器，发到peerB(即：网络协商)

这样peerA，peerB就相互交换了媒体信息及网络信息，就可以开始通讯了。

WebRTC环境配置



luge@localhost:/home/luge

```
0: relay 0.0.0.0 initialization...
0: relay 0.0.0.0 initialization done
0: Relay ports initialization done
0: IO method (general relay thread): epoll (with changelist)
0: turn server id=0 created
0: IO method (general relay thread): epoll (with changelist)
0: turn server id=1 created
0: IO method (general relay thread): epoll (with changelist)
0: turn server id=2 created
0: IO method (general relay thread): epoll (with changelist)
0: turn server id=3 created
0: IO method (general relay thread): epoll (with changelist)
0: turn server id=4 created
0: IO method (general relay thread): epoll (with changelist)
0: turn server id=5 created
0: IO method (general relay thread): epoll (with changelist)
0: turn server id=6 created
0: IO method (general relay thread): epoll (with changelist)
0: turn server id=7 created
0: IPv4. UDP listener opened on: 0.0.0.0:3478
0: Total General servers: 8
0: IO method (auth thread): epoll (with changelist)
0: IO method (auth thread): epoll (with changelist)
0: IO method (auth thread): epoll (with changelist)
0: IO method (auth thread): epoll (with changelist)
0: IPv4. SCTP listener opened on: 0.0.0.0:3478
0: IO method (admin thread): epoll (with changelist)
0: IPv4. TCP listener opened on: 0.0.0.0:3478
0: IPv4. CLI listener opened on: 127.0.0.1:5766
0: IPv4. TCP listener opened on: 0.0.0.0:3478
0: IPv4. TCP listener opened on: 0.0.0.0:3478
0: IPv4. TCP listener opened on: 0.0.0.0:3478
0: IPv4. TCP listener opened on: 0.0.0.0:3478
0: IPv4. TCP listener opened on: 0.0.0.0:3478
0: IPv4. TCP listener opened on: 0.0.0.0:3478
33: handle_udp_packet: New UDP endpoint: local addr 0.0.0.0:3478, remote addr 192.168.0.104:56202
33: handle_udp_packet: New UDP endpoint: local addr 0.0.0.0:3478, remote addr 192.168.0.104:56206
33: handle_udp_packet: New UDP endpoint: local addr 0.0.0.0:3478, remote addr 192.168.0.104:56204
33: session 00100000000000000001: realm <nort.gov> user <>: incoming packet BINDING processed, success
33: session 00200000000000000001: realm <nort.gov> user <>: incoming packet BINDING processed, success
33: session 00500000000000000001: realm <nort.gov> user <>: incoming packet BINDING processed, success
3
```

luge@localhost:/home/luge 1/4

released to the application can be controlled via the IceTransports constraint.

If you test a STUN server, it works if you can gather a candidate with type 'srflx'. If you test a TURN server, it works if you can gather a candidate with type 'relay'.

If you test just a single TURN/UDP server, this page even allows you to detect when you are using the wrong credential to authenticate.

ICE servers

stun:192.168.0.105:3478 [root:123456]

STUN or TURN URI:

TURN username:

TURN password:

ICE options

IceTransports value: ☒ all ☐ relay

ICE Candidate Pool:

Time	Component	Type	Foundation	Protocol	Address	Port	Priority
0.001		host	4633732497	udp	7f9e8b495f42e8-8d58-3d5087b5c155 local	56769	128 0 0
0.102							Done
0.103							

Note: errors from onicecandidateerror above are not necessarily fatal. For example an IPv6 DNS lookup may fail but relay candidates can still be gathered via IPv4.

[View source on GitHub](#)

更换centos的openssl的版本

openssl version

wget https://www.openssl.org/source/openssl-1.1.1c.tar.gz

tar -zxvf openssl-1.1.1c.tar.gz

cd openssl-1.1.1c

./config --prefix=/usr/local/openssl #如果此步骤报错,需要安装perl以及gcc包

make && make install

mv /usr/bin/openssl /usr/bin/openssl.bak

ln -sf /usr/local/openssl/bin/openssl /usr/bin/openssl

echo "/usr/local/openssl/lib" >> /etc/ld.so.conf

ldconfig -v # 设置生效

openssl version

注意你更换了新的版本需要把yum安装的老的版本卸载掉 否则在编译的时候还是链接的老版本

yum remove openssl-devel

启动服务

```
nohup turnserver -L 0.0.0.0 -a -u root:123456 -v -f -r nort.gov &
```

#<外网ip>:3478

#check

<https://webrtc.github.io/samples/src/content/peerconnection/trickle-ice/>

本地音视频采集浏览器标准API

<https://developer.mozilla.org/zh-CN/docs/Web/API/MediaDevices>

核心API理解:

MediaDevices接口提供访问连接媒体输入的设备,如照相机和麦克风,以及屏幕共享等。它可以使你取得任何硬件资源的媒体数据。

getSupportedConstraints()

约束: 可以获取浏览器的UA所支持的约束。(aspectRatio、autoGainControl、brightness、deviceId等)

MediaDevices.getUserMedia() 在用户通过提示允许的情况下,打开系统上的相机或屏幕共享和/或麦克风,并提供 **MediaStream** 包含视频轨道和/或音频轨道的输入。**MediaStream**可以获取音频轨道信息。

MediaDevices.enumerateDevices() 请求一个可用的媒体输入和输出设备的列表

实战代码: 参见直播课上分析。

信令专题

网易云信分析



https://github.com/netease-im/Signaling_Sample_Code/tree/master/Android

记住几句话:

- WebRTC是没有将信令服务纳入到整个的规范中，更多的是规范了客户端这边的所有的过程。（每个公司的业务不一样，所以要自己去处理）
- 我们在通话或者视频的时候，需要传递的信息包括网络和媒体信息，媒体信息我们通过SDP来协商，网络通过P2P的形式进行传输。
- 业务的独特信息，包括房间管理、禁言、权限管理都需要进行交换。
- 从技术选型的角度上说，对于信令而言，一定要保证传输的可达性，TCP作为底层的协议保障是必不可少的。

信令底层技术理论

Socket.IO

<https://socket.io/>

<https://socket.io/docs/v4>

socket.io是一个跨浏览器支持WebSocket的实时通讯的JS。

Socket.io支持及时、双向、基于事件的交流，可在不同平台、浏览器、设备上工作，可靠性和速度稳定。最典型的应用场景如：

- 实时分析：将数据推送到客户端，客户端表现为实时计数器、图表、日志客户。
- 实时通讯：聊天应用
- 二进制流传输：socket.io支持任何形式的二进制文件传输，例如图片、视频、音频等。
- 文档合并：允许多个用户同时编辑一个文档，并能够看到每个用户做出的修改。

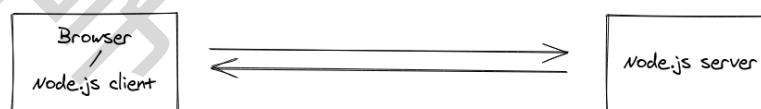
Socket.io实际上是WebSocket的父集，Socket.io封装了WebSocket和轮询等方法，会根据情况选择方法来进行通讯。

Node.js提供了高效的服务端运行环境，但由于Browser对HTML5的支持不一，为了兼容所有浏览器，提供实时的用户体验，并为开发者提供客户端与服务端一致的编程体验，于是Socket.io诞生了。

What Socket.IO is

Socket.IO is a library that enables real-time, bidirectional and event-based communication between the browser and the server. It consists of:

- a Node.js server: [Source](#) | [API](#)
- a javascript client library for the browser (which can be also run from Node.js): [Source](#) | [API](#)



There are also several client implementation in other languages, which are maintained by the community:

- Java: <https://github.com/socketio/socket.io-client-java>
- C++: <https://github.com/socketio/socket.io-client-cpp>
- Swift: <https://github.com/socketio/socket.io-client-swift>
- Dart: <https://github.com/rikulo/socket.io-client-dart>
- Python: <https://github.com/miguelgrinberg/python-socketio>
- .Net: <https://github.com/doghappy/socket.io-client-csharp>
- Golang: <https://github.com/googollee/go-socket.io>
- Rust: <https://github.com/1c3t3a/rust-socketio>

Other server implementations:

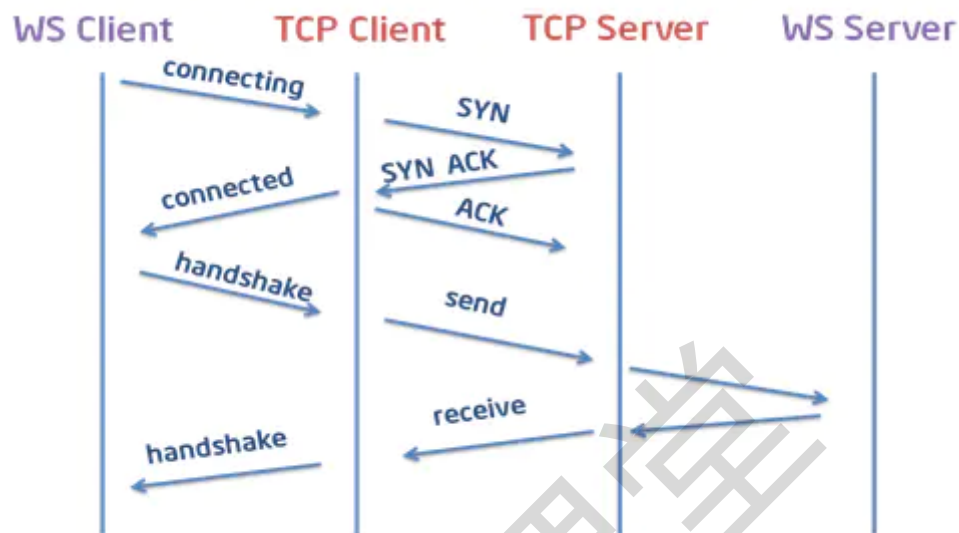
- Java: <https://github.com/mrniko/netty-socketio>
- Java: <https://github.com/trinopoty/socket.io-server-java>
- Python: <https://github.com/miguelgrinberg/python-socketio>

了解一个基本的概念：**就是Socket.IO和WebSocket的区别和联系：**

WebSocket 原理

WebSocket是一种**双向通信协议**，它建立在TCP之上，同HTTP一样通过TCP来传输数据，但与HTTP最大不同的是：

- WebSocket是一种双向通信协议，在建立连接后，WebSocket服务器和Browser/UserAgent都能主动的向对象发送或接收数据，就像Socket一样，不同的是WebSocket是一种建立在Web基础上的简单模拟Socket的协议。
- WebSocket需要通过握手连接，类似TCP也需要客户端和服务端进行握手连接，连接成功后才能相互通信。



信令服务器的设计思想

https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API/Signaling_and_video_calling#the_signaling_server

https://developer.mozilla.org/zh-CN/docs/Web/API/WebRTC_API/Protocols

First we need the signaling server itself. WebRTC doesn't specify a transport mechanism for the signaling information. You can use anything you like, from WebSocket to XMLHttpRequest to carrier pigeons to exchange the signaling information between the two peers.

It's important to note that the server doesn't need to understand or interpret the signaling data content. Although it's SDP, even this doesn't matter so much: the content of the message going through the signaling server is, in effect, a black box. What does matter is when the ICE subsystem instructs you to send signaling data to the other peer, you do so, and the other peer knows how to receive this information and deliver it to its own ICE subsystem. All you have to do is channel the information back and forth. The contents don't matter at all to the signaling server.

核心要点总结：

Exchanging session descriptions: offer 和 answer 机制

当启动信令过程时，由发起呼叫的用户创建一个offer。此服务包括SDP格式的会话描述，需要传递给接收用户，我们称之为被叫方。被叫方以应答消息回应该提议，该消息还包含SDP描述。我们的信令服务器将使用WebSocket传输类型为“video offer”的offer消息，并应答类型为“video answer”的消息。这些消息包含以下字段：

type

The message type; either "video-offer" or "video-answer".

name

The sender's username.

target

The username of the person to receive the description (if the caller is sending the message, this specifies the callee, and vice-versa).

sdp

The SDP (Session Description Protocol) string describing the local end of the connection from the perspective of the sender (or the remote end of the connection from the receiver's point of view).

[Exchanging ICE candidates](#)

两个对等方需要交换ICE候选方来协商它们之间的实际连接。每个ICE候选者描述了发送对等方能够用来通信的方法。每个对等方都会按照发现候选对象的顺序发送候选对象，并不断发送候选对象，直到建议用完为止，即使媒体已经开始流式传输。

type

The message type: "new-ice-candidate".

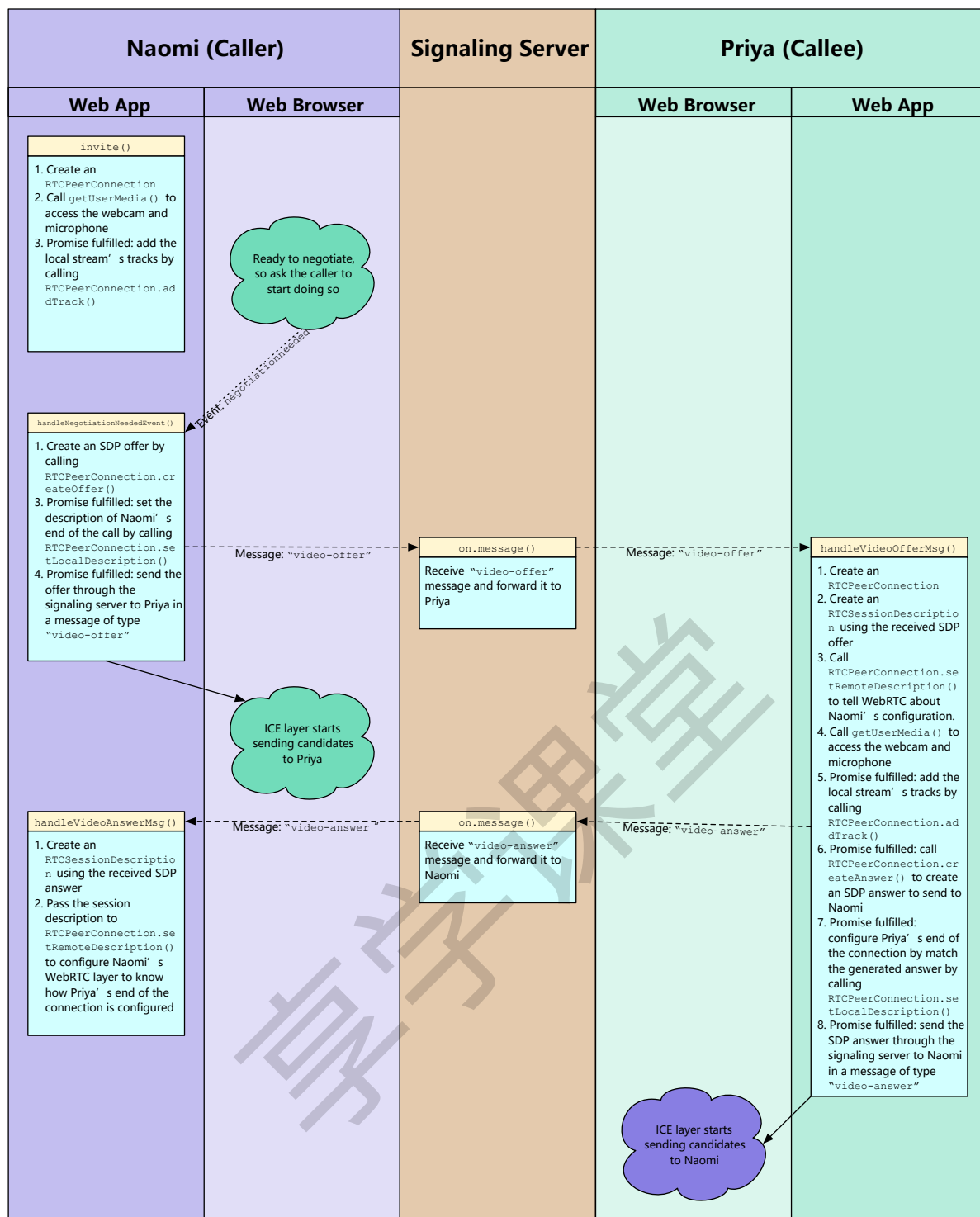
target

The username of the person with whom negotiation is underway; the server will direct the message to this user only.

candidate

The SDP candidate string, describing the proposed connection method. You typically don't need to look at the contents of this string. All your code needs to do is route it through to the remote peer using the signaling server.

[Signaling transaction flow](#) (重要的流程，需要理解自己手绘制)



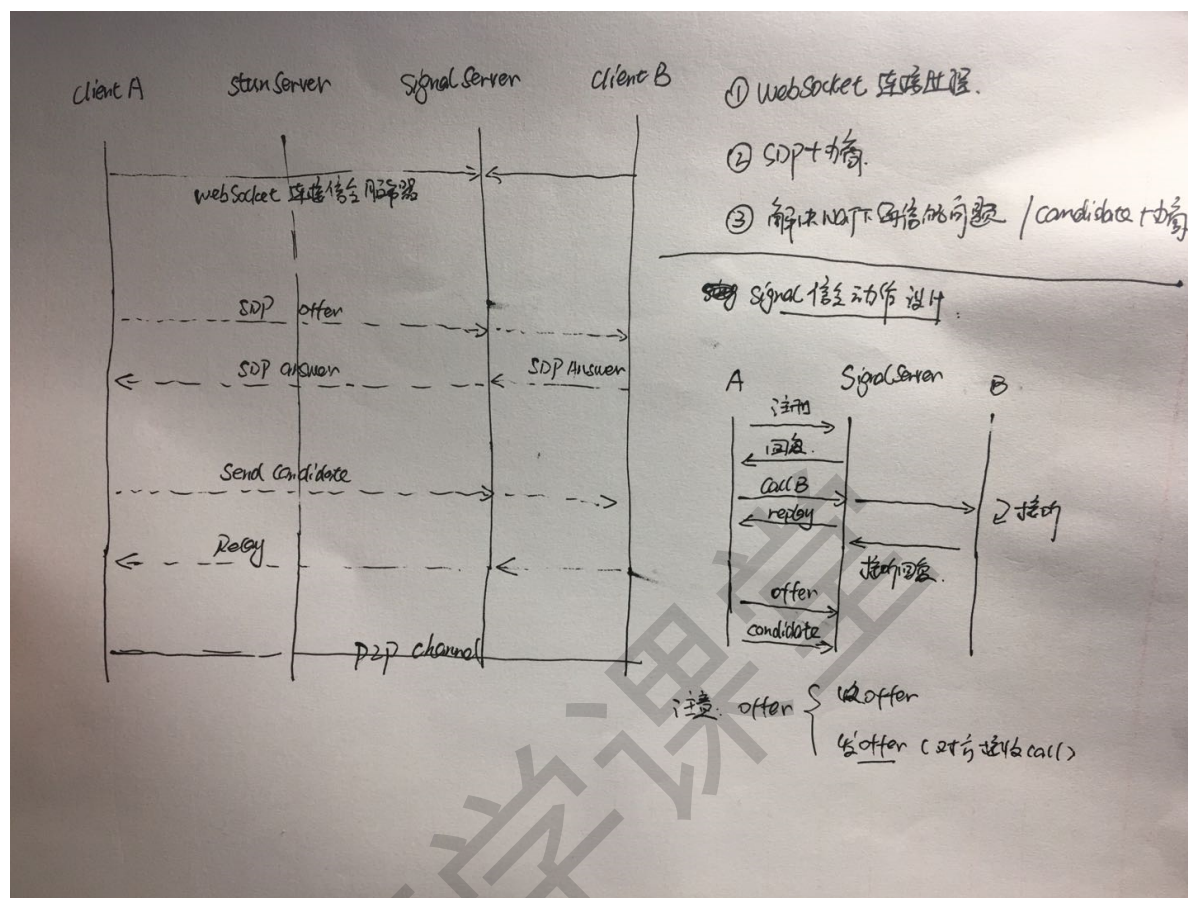
ICE candidate exchange process

WebRTC1v1音视频通话代码实践

<https://webrtc.github.io/samples/>

考虑到大家主要是android，接入部分我将采用android/java来实现，我们也可以用web来完成。

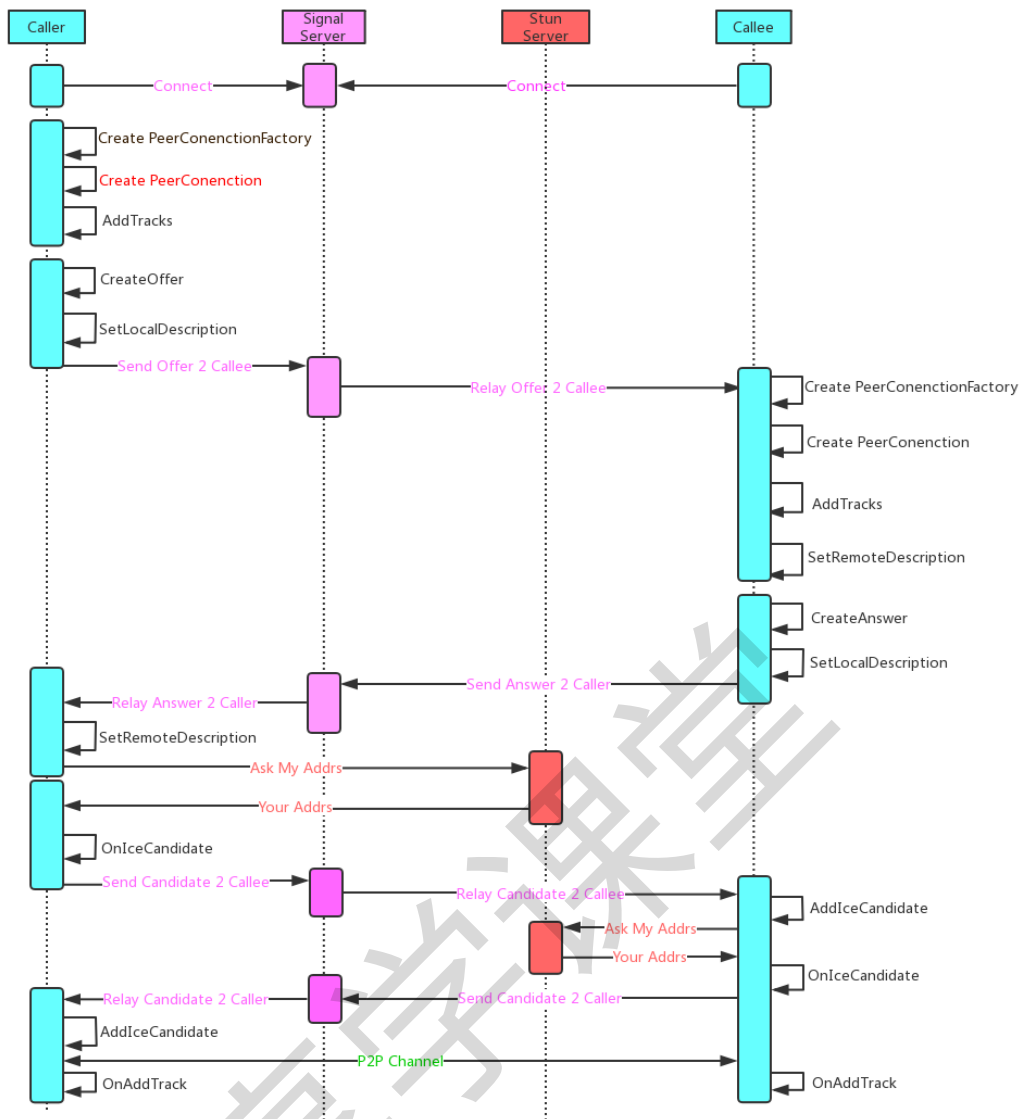
信令设计



WebRTC PeerConnection 建链

通信过程中的基本概念:

- Room: Signaling Server 使用 Room 的概念对一组通信的 peers 进行管理，一个 Room 中包含一个或多个 peers，当没有 peers 存在时 Room 自动注销；当第一个 peer 连接到 Signaling Server 时执行 Create Room 操作；
- Offer/Answer: 描述 peer 媒体属性的 SDP 信息通过 Offer/Answer 模式，由 Signaling Server 进行交换，通信建立前两个互相通信的 peer 需要互相获得对方的媒体 SDP 信息；
- IceCandidate: Peer 与 STUN/TURN/ICE Server 通信，获取自己的 NAT 类型以及公网 IP 和端口，这些消息简称为 IceCandidate。和媒体描述信息一样，IceCandidate 信息也需要互相通信的 peer 在通信前进行交换，在获取到对方的 iceCandidate 信息后双方通过 Stun binding 信令确认信道的联通性；



参考资料: <https://www.w3.org/TR/webrtc/>

https://datatracker.ietf.org/doc/draft-nandakumar-rtcweb-sdp/?include_text=1

<https://webrtc.googlesource.com/src/+master/examples>